AI Engineering: Complete Step-by-Step Roadmap (2026)

Since you already have experience with Python, SQL, FastAPI, and some Generative AI, you don't need to start from zero. Your goal should be to become an AI Engineer who can build, deploy, and maintain production-ready AI applications, rather than only learning how to call an LLM API.

This roadmap takes you from your current foundation to job-ready AI engineering, including traditional machine learning, deep learning, LLMs, RAG, AI agents, deployment, projects, and interview preparation.

1. The complete roadmap at a glance

Phase	Weeks	Focus
1 — Python and software engineering	1–2	Python, OOP, typing, async programming, Git, testing, APIs and SQL.
2 — Mathematics, NumPy and data	3–4	Statistics, probability, linear algebra, NumPy, pandas and visualization.
3 — Machine learning	5–7	Supervised learning, unsupervised learning, model evaluation and scikit-learn.
4 — Deep learning	8–9	Neural networks, backpropagation, PyTorch, embeddings and Transformers.
5 — Generative AI and LLMs	10–12	LLM APIs, prompting, structured outputs, embeddings, RAG and evaluation.
6 — AI agents and workflows	13–14	Tool calling, LangGraph, state, memory, human approval and agent evaluation.
7 — Production AI engineering	15–16	Docker, deployment, monitoring, security, cost optimization and CI/CD.
8 — Portfolio and job preparation	17–20	Three strong projects, GitHub documentation, interview preparation and applications.

Timeline assumes roughly 3–4 focused hours daily. It's an intensive target, not a guarantee of job readiness; your existing skills may shorten the initial phases.

2. Phase 1: Python and software engineering foundations

Weeks 1–2

You already know intermediate Python, so focus on writing reliable, maintainable code rather than repeating beginner tutorials.

Topics to master

Area	What to learn
Python core	Data types, comprehensions, functions, decorators, generators, iterators
OOP	Classes, inheritance, composition, abstraction, polymorphism
Type hints	`list[str]`, `dict`, `Optional`, `TypedDict`, Pydantic models
Error handling	Exceptions, custom exceptions, logging
Async programming	`async`, `await`, coroutines, concurrency
Software design	Modules, packages, clean architecture, SOLID basics
Testing	pytest, fixtures, mocking, API tests

Area	What to learn
Developer tools	Git, GitHub, virtual environments, uv, environment variables

Also revise HTTP methods, REST APIs, JSON, authentication, SQL joins, indexes and transactions.

Practical tasks

- Build a FastAPI CRUD API with PostgreSQL.
- Add Pydantic validation and centralized exception handling.
- Write pytest tests for success and failure cases.
- Add logging and environment-based configuration.

Deliverable: a tested, well-structured Python API that you can reuse as the backend for future AI projects.

Resource: Official FastAPI tutorial.

3. Phase 2: Mathematics, NumPy and data processing

Weeks 3–4

You don't need to become a mathematician. You need enough mathematical understanding to explain how models learn and how to interpret their outputs.

A. Mathematics

- **Linear algebra:** vectors, matrices, matrix multiplication, dot products, norms and cosine similarity.
- **Probability and statistics:** mean, median, variance, standard deviation, probability distributions, conditional probability and Bayes' theorem.
- **Calculus and optimization:** derivatives, partial derivatives, gradients, chain rule, gradient descent and loss functions.

B. Data libraries

Learn these in order:

1. **NumPy:** arrays, shapes, broadcasting, indexing, vectorization and matrix operations.
2. **pandas:** loading files, missing values, duplicates, joins, grouping, aggregation and feature engineering.
3. **Matplotlib / Seaborn:** distributions, correlations, scatter plots, outliers and model-performance visualizations.

Practice project

Analyze an e-commerce dataset and identify:

- Most profitable products.
- Customer segments.
- Factors associated with repeat purchases.
- Relationships between discounts and sales.

Don't just create charts. Explain what each finding means and what business decision it supports.

4. Phase 3: Machine learning

Weeks 5–7

Machine learning is important because it teaches you how to build and evaluate predictive systems, even when an LLM isn't the right solution.

A. Learn the major algorithms

Category	Algorithms
Regression	Linear regression, ridge, lasso
Classification	Logistic regression, decision trees, random forest, SVM, k-NN
Boosting	Gradient boosting, XGBoost or LightGBM
Clustering	K-means, hierarchical clustering, DBSCAN
Dimensionality reduction	PCA
Anomaly detection	Isolation Forest

You don't need to implement every algorithm from scratch. Understand the intuition, assumptions, use cases and trade-offs.

B. Learn the complete ML workflow

```
Raw dataset
    ↓
Data validation and exploration
    ↓
Data cleaning and preprocessing
    ↓
Train / validation / test split
    ↓
Feature engineering
    ↓
Train baseline model
    ↓
Evaluate and tune
    ↓
Error analysis
    ↓
Save model and expose an API
    ↓
Monitor performance
```

Key concepts:

- Overfitting vs. underfitting.
- Data leakage.
- Cross-validation.
- Feature scaling and encoding.
- Hyperparameter tuning.
- Class imbalance.
- Precision, recall, F1-score and ROC-AUC.
- MAE, MSE and RMSE for regression.
- Model interpretability and feature importance.

Practical project: Customer churn prediction

Build a model that predicts which customers are likely to stop using a service. Use Python, pandas, scikit-learn and FastAPI. Evaluate the model, analyze incorrect predictions and expose a prediction endpoint.

Resource: Scikit-learn Getting Started. It covers fitting models, preprocessing, pipelines, model selection and evaluation.

5. Phase 4: Deep learning and neural networks

Weeks 8–9

Learn the fundamentals before jumping into advanced architectures.

Topics

- Perceptrons and neural network layers.
- Activation functions: ReLU, sigmoid, softmax.
- Forward propagation and backpropagation.
- Loss functions and optimizers.
- Gradient descent and learning rates.
- Batch size, epochs and regularization.
- Embeddings and tensor operations.
- GPU basics and model checkpoints.

Learn PyTorch

Focus on:

1. Tensors and tensor shapes.
2. Datasets and DataLoaders.
3. Building models using `torch.nn`.
4. Automatic differentiation with `autograd`.
5. Training and validation loops.
6. Saving and loading models.
7. Debugging training and evaluating results.

Use the official PyTorch beginner tutorials, which walk through a complete training workflow.

Understand these architectures

- Feed-forward neural networks.
- CNNs for image-related tasks.
- RNNs and LSTMs as historical foundations for sequence modeling.
- Attention mechanisms.
- Transformers.

Project: Train a sentiment classifier on customer reviews and serve it through FastAPI.

You can keep this phase relatively short if your immediate goal is an LLM application engineering role. Don't spend months trying to master every deep learning architecture before building with LLMs.

6. Phase 5: Generative AI and Large Language Models (LLMs)

Weeks 10–12

This is one of the most important phases for your target role.

A. Understand how LLMs work

Study the following concepts in order:

1. NLP fundamentals and text tokenization.
2. Tokens, vocabulary and token IDs.
3. Embeddings and semantic similarity.
4. Self-attention and positional information.
5. Transformer architecture.
6. Pretraining and next-token prediction.
7. Inference and context windows.
8. Temperature, top-p and sampling.
9. Hallucinations and model limitations.
10. Quantization, inference cost and latency.

You should be able to explain, at a high level, how a Transformer uses attention and why an LLM can generate fluent but incorrect information.

B. Work with LLM APIs

Start with one provider, such as Groq or Gemini, and learn transferable patterns.

- System, user and developer instructions.
- Prompt design and prompt templates.
- Structured output with JSON schemas.
- Tool/function calling.
- Streaming responses.
- Retry logic and rate limits.
- Context management.
- Model selection and cost comparison.
- API key security.

Build small experiments before introducing a framework.

C. Learn Hugging Face

Understand the ecosystem:

- `transformers`
- `datasets`
- `tokenizers`
- Model and dataset hubs.
- Pipelines and model inference.
- Basic fine-tuning and evaluation.

Resource: Hugging Face LLM course. It is a free, open-source learning resource for Transformers and the surrounding ecosystem.

D. Learn fine-tuning concepts

Learn these after you understand inference and prompting:

- Pretraining vs. fine-tuning.
- Supervised fine-tuning (SFT).
- LoRA and QLoRA.
- Parameter-efficient fine-tuning.
- Dataset preparation and quality.

- Training and evaluation splits.
- When to fine-tune vs. use RAG.

You do not need to train a foundation model from scratch.

Practical project

Create an AI resume analyzer that accepts a CV and job description, extracts skills, compares requirements, returns structured results and explains missing qualifications. Use an LLM for reasoning and extraction, Pydantic for validation, and FastAPI for the API.

7. Phase 6: Embeddings, vector databases and RAG

Weeks 11–13

RAG (Retrieval-Augmented Generation) is a core technique for building AI systems that answer questions using external documents or private business data.

Understand the RAG pipeline

Step	Description
1. Ingest documents	PDF, DOCX, CSV, web pages, databases
2. Clean and chunk	Split documents into useful text segments
3. Generate embeddings	Convert text into numerical vectors
4. Store and retrieve	Vector search, metadata filters, reranking
5. Generate a grounded answer	Provide retrieved context to an LLM and return citations

Topics to learn

- Embedding models and cosine similarity.
- Chunk size, overlap and metadata.
- Vector search vs. keyword search.
- Hybrid search.
- Metadata filtering.
- Reranking.
- Context construction.
- Retrieval precision and recall.
- RAG evaluation and hallucination reduction.
- Document access controls and source citations.

Recommended tools

Purpose	Technologies
Embeddings	Sentence Transformers, provider embedding APIs
Vector storage	pgvector, Qdrant, Chroma
Document parsing	PyMuPDF, python-docx, pandas
Retrieval workflows	LangChain or custom Python
API	FastAPI
Persistence	PostgreSQL

Learn the underlying retrieval process before relying heavily on a framework.

Project: Build a document intelligence assistant that lets users upload files, ask questions across multiple documents, receive source citations and manage their document collections.

A strong version should handle incorrect questions, missing evidence, document updates and access permissions.

8. Phase 7: AI agents and multi-step workflows

Weeks 13–14

An LLM application answers or generates content. An AI agent can also select tools, execute actions and decide what step to take next within defined limits.

Learn these concepts

1. Tool calling and function schemas.
2. Workflow orchestration.
3. State management.
4. Short-term and persistent memory.
5. Conditional branching and loops.
6. Retries, timeouts and failure recovery.
7. Human-in-the-loop approvals.
8. Multi-agent coordination.
9. Tracing, evaluations and cost controls.
10. Guardrails and permissions.

Learn LangGraph

Use it when you need stateful workflows, explicit control flow, checkpointing or human approval. Learn the basic workflow patterns before building a complicated multi-agent system.

Resource: LangChain and LangGraph learning guides. These include tutorials for RAG, SQL agents, data analysis agents and custom workflows.

Practical project: AI Data Analyst Agent

This is an excellent flagship project for your background in Python, SQL, pandas and FastAPI.

Stage	Description
1. Data ingestion	Accept CSV, Excel and supported database connections.
2. Data analysis tools	Use pandas and SQL to profile data, calculate metrics and validate results.
3. Natural-language questions	Ask questions such as “Which products had the highest revenue growth?”
4. Explainable reports	Generate charts, concise findings, metric definitions and evidence from the underlying data.

Important: don't let the LLM freely execute arbitrary Python or SQL. Validate queries, restrict database permissions, isolate code execution when necessary, and require approval for consequential operations.

9. Phase 8: Production AI engineering and MLOps

Weeks 15–16

Knowing how to build a working demo is different from knowing how to operate an AI product reliably.

A. Backend and architecture

- FastAPI routers, dependencies and middleware.
- Pydantic request and response schemas.
- PostgreSQL and SQLAlchemy.
- Authentication and role-based access.
- Background jobs and task queues.
- Streaming responses and WebSockets.
- Caching with Redis.
- File upload validation and storage.
- API rate limiting and request timeouts.

B. Deployment and infrastructure

- Git and GitHub.
- Docker and Docker Compose.
- Environment variables and secrets.
- CI/CD with GitHub Actions.
- Cloud deployment.
- Logging and error monitoring.
- Database migrations and backups.
- Basic Linux and networking.

C. AI-specific production skills

Area	What you should know
Evaluation	Test datasets, groundedness, correctness and task success
Observability	Track prompts, latency, token usage, failures and tool calls
Reliability	Retries, fallbacks, timeouts and graceful degradation
Security	Prompt injection, PII handling, access controls and secrets
Cost	Token budgets, caching, batching and model selection
Performance	Streaming, concurrency, retrieval latency and load testing
Model operations	Model versions, dataset versions, drift and rollback where applicable

Learn the principles first; tools such as MLflow, Langfuse, OpenTelemetry and experiment-tracking platforms can be added when a project needs them.

Deliverable: one AI application deployed publicly, with authentication, tests, monitoring, clear setup instructions and documented limitations.

10. Phase 9: Advanced topics — learn after the fundamentals

These skills can differentiate you later, but they shouldn't delay your first job application.

Topic	Key concepts
Advanced RAG	Query rewriting, hybrid retrieval, rerankers, graph-based retrieval
Fine-tuning	LoRA/QLoRA, SFT, dataset curation and evaluation
Model serving	vLLM, batching, quantization and throughput
Multimodal AI	Vision-language models, audio, speech and image understanding

Topic	Key concepts
AI safety	Prompt injection defenses, output validation and red teaming
Distributed systems	Queues, caching, scaling, asynchronous workloads
Advanced ML	Experiment tracking, model registries, drift monitoring
AI research literacy	Read papers, reproduce experiments and understand benchmarks

Learn Kubernetes, distributed training and advanced multi-agent systems only when your target roles or project requirements justify them.

11. Build these three portfolio projects

Don't build ten basic chatbots. Build three complete applications that demonstrate different AI engineering skills.

Project 1 — AI Data Analyst Agent (highest priority)

Demonstrates SQL, pandas, LLMs, tool calling, data validation and visualization.

Stack: Python, FastAPI, PostgreSQL, pandas, LangGraph, React/Next.js.

Project 2 — Enterprise Document Intelligence

Demonstrates ingestion, embeddings, hybrid retrieval, citations, RAG evaluation and access controls.

Stack: Python, FastAPI, PostgreSQL/pgvector, Sentence Transformers, an LLM API.

Project 3 — Production-Ready AI Interview Coach

Demonstrates audio processing, resume parsing, structured feedback, model evaluation and real-time user interaction.

Stack: Next.js, FastAPI, PostgreSQL, speech-to-text, text-to-speech and an LLM.

You already have an InterviewAI project, so improve that project instead of starting another mock-interview app from scratch. Prioritize the first two projects to show that you can solve different classes of problems.

For every project, include:

- A deployed demo, if feasible.
- A clean GitHub repository and architecture diagram.
- A detailed README with installation instructions.
- Tests and sample inputs/outputs.
- Evaluation results on a defined test set.
- Screenshots or a short demo video.
- Honest documentation of limitations, latency and cost.

12. Your daily study schedule

For a focused 3–4 hour daily routine, use this template. Use the schedule as a starting point. During the first few weeks, allocate more time to fundamentals; during project weeks, increase implementation time.

Activity	Time (3-hour steady pace, balanced focus)
Concepts and documentation	60 minutes
Hands-on coding	108 minutes

Activity	Time (3-hour steady pace, balanced focus)
Project development	48 minutes
Revision and interview prep	24 minutes

Available pacing options: 2 hours (slower, part-time), 3 hours (steady), 4 hours (intensive) and 6 hours (full-time study). Focus options: Balanced (learning, coding and interview prep) or Job-first (projects, applications and interviews).

13. Free learning resources

These resources are enough to begin without buying an expensive course.

Area	Resource
Python and backend development	FastAPI official tutorial — APIs, validation, dependencies, security and testing.
Machine learning	Scikit-learn guide — model training, preprocessing and evaluation.
Deep learning	PyTorch tutorials — tensors, neural networks, training and model saving.
LLMs and Transformers	Hugging Face course — Transformers, datasets, tokenizers and fine-tuning.
RAG and AI agents	LangChain and LangGraph guides — RAG, SQL agents and stateful workflows.

14. AI Engineering interview preparation

Prepare for interviews throughout the roadmap rather than waiting until you finish it.

Interview area	What to prepare
Python	OOP, generators, decorators, async/await, exceptions
DSA	Arrays, strings, hash maps, stacks, queues, recursion, trees, complexity
SQL	Joins, CTEs, window functions, aggregations, indexes and query optimization
Machine learning	Bias/variance, data leakage, metrics, overfitting, model selection
Deep learning	Backpropagation, loss functions, optimizers, attention and Transformers
Generative AI	Tokens, embeddings, prompting, RAG, fine-tuning and hallucinations
AI agents	Tool calling, state, workflows, retries, memory and human approval
Backend	FastAPI, authentication, databases, API design and testing
System design	Scalability, caching, queues, rate limits, observability and reliability
Project discussion	Architecture, trade-offs, evaluation, failures, security and deployment

Also prepare to explain why you chose a particular model, how you evaluated its quality, how you would reduce hallucinations, and what happens when an external model API becomes unavailable.

15. Your first 30 days: an actionable plan

Since you want to enter the job market quickly, follow this initial plan rather than waiting until you finish every topic in the full roadmap.

30-day execution checklist

Week	Theme	Tasks
Week 1	Strengthen Python and APIs	☐ Revise Python typing, OOP and async/await ☐ Practice SQL joins and window functions ☐ Build a tested FastAPI endpoint

Week	Theme	Tasks
Week 2	Learn practical ML	☐ Revise statistics and data preprocessing ☐ Train a scikit-learn classification model ☐ Evaluate results and document errors
Week 3	Build an LLM application	☐ Use an LLM API with structured outputs ☐ Implement document parsing and embeddings ☐ Build a basic RAG workflow with citations
Week 4	Deploy and apply	☐ Add validation, tests and error handling ☐ Deploy the application and document it ☐ Update resume and GitHub; apply to suitable roles

Start applying before day 30 if you can already demonstrate the required skills. Search for Junior AI Engineer, Generative AI Engineer, Python Developer (AI), LLM Application Engineer, AI Software Engineer, and Associate Machine Learning Engineer roles. Requirements differ by company, so tailor each application accordingly.

My recommendation

Your best route is to combine your existing Python and FastAPI skills with LLM application development, RAG, evaluation and production deployment. Learn enough traditional ML and PyTorch to understand the foundations, but prioritize building reliable AI applications that solve real problems.

You don't have to master every advanced topic before applying. A strong deployed project, a clear understanding of the architecture and consistent interview preparation can be more valuable than completing dozens of courses.